07
Design A Key-value Store

A key-value store, also referred to as a key value database, is a non-relational database. Each unique identifier is stored as a key with its associated value. This data pairing is known as a "key-value" pair.

In a key-value pair, the key must be unique, and the value associated with the key can be accessed through the key. Keys can be plain text or hashed keys. For performance reasons, a short key works better. What do keys look like? Here are a few examples:

- Plain text key: "last_logged_in_at"
- Hashed key: "253DDC1116"

The value in a key-value can be string, lists, objects, etc. The value in a key-value is usually treated as an opaque object in key-value stores and is not interpreted by the key-value store.

Here's a table snippet in key-value format:

key	value
100	John
101	Jack
102	Jill

In this chapter, you are tasked to design a key-value store that supports the following operations:

- putKey(key, value) // insert "value" associated with "key"
- getValue(key) // get "value" associated with "key"

Understand the problem and establish design scope

There is no perfect design for a key-value store. Each design must strike a balance between read, write, and memory usage and achieve a high degree of scalability, latency, and consistency. Fortunately, in an interview, you do not have to cover every single detail. You should clarify how much detail you need to provide. In reality, no design is ever done but there comes a time when it is ready to be released. In this chapter, we discuss what goes into the design of a key-value store that is small in size, but it works well.

- The size of key-value pair is small: less than 10 KB
- Ability to store big data.
- High scalability. The system responds quickly, even during traffic.
- High availability. The system can be queried to support large data set.
- Tunable consistency. Consistency is a tunable option of server should be queried based on traffic.
- Low latency.
- Low memory footprint.
- Low consistency.

Single server key-value store

Developing a key-value store that resides on a single server is easy. An intuitive approach is to store key-value pairs in a hash table, which guarantees fast read and write. Although it is possible to store everything in memory, it may not be impossible due to space restriction.

To achieve high availability, we can store multiple copies of the data on a single server.

- Store key-value pairs in memory
- Store key-value pairs in disk

Even with those optimizations, a single server can reach its capacity very quickly. A distributed key-value store is required to support big data.

Distributed key-value store

A distributed key-value store is also called a distributed hash table, which distributes key-value pairs across many servers. When designing a distributed system, it is important to understand CAP (Consistency, Availability, Partition Tolerance) theorem.

CAP theorem

CAP theorem states it is impossible for a distributed system to simultaneously provide more than two of these three guarantees: consistency, availability, and partition tolerance. Let us establish a few definitions.

Consistency: consistency means all clients see the same data at the same time no matter which node they connect to.

Availability: availability means any client which requests data gets a response even if some of the nodes are down.

Partition Tolerance: a partition denotes a communication break between two nodes. Partition tolerance means the system continues to operate despite network partitions.

CAP theorem states that any of the three properties must be sacrificed to support 2 of the 3 properties as shown in Figure 1.

Figure 1

Nowadays, key-value stores are classified based on the two CAP characteristics they support.

CP (consistency and partition tolerance) systems. A CP key-value store supports consistency and partition tolerance while sacrificing availability.

AP (availability and partition tolerance) systems. An AP key-value store supports availability and partition tolerance while sacrificing consistency.

CA (consistency and availability) systems. A CA key-value store supports consistency and availability while sacrificing partition tolerance. Since network failure is unavoidable, a distributed system must tolerate network partition. Thus, a CA system cannot exist in real-world applications.

What you read above is mostly the definitions part. It makes it easier to understand but can be a little vague in real-world scenarios. In distributed systems, data is usually replicated multiple times. Assume data are replicated on three replica nodes, n1, n2 and n3 as shown in Figure 2.

Ideal situation

In the ideal world, network partition never occurs. Data written to n1 is automatically replicated to n2 and n3. Both consistency and availability are achieved.

Figure 2

In real life, network partitions will happen. In a distributed system, we must decide either to compromise consistency or availability in the event of a network partition. Imagine a network failure has made the connection between n1 and n2 unreachable. Two replicas n1 and n2 communicate to n3. If data is written to n1 but not n2, n2 will have stale data. If n1 and n2 are kept online, inconsistency occurs.

Figure 3

If we choose consistency over availability (CP system), we must delay all write operations to n1 and n2 to avoid data inconsistency among these two replicas. Depending on how long the network partition takes place, the delayed write operations can be queued. Availability is compromised as we wait for the network partition to be fixed. Once the network partition is resolved, the data on n1 can be safely propagated to n2, and the queued write operations can be executed to bring n2 up to speed. In a network partition, the queued operations will delay the system response and cause some unavailability.

If instead, if we choose availability over consistency (AP system), the system can keep accepting reads, even though it might return stale data. For writing, n1 and n2 will keep accepting writes, and data will be synced to n3 when the network partition is resolved.

Choosing the right CAP combination for a distributed key-value store is important. Depending on business requirements, a careful assessment of the advantages and disadvantages for each combination must be made. For example, in a financial system, returning stale data is unacceptable because it could be the difference between life and death.

System components

In this section, we will discuss the following core components and techniques used to build a key-value store.

- Data partition
- Data replication
- Consistency
- Inconsistency resolution
- Handling failures
- System architecture diagram
- Write path
- Read path

The components to implement are mostly based on three popular key-value store systems: Dynamo [3], Cassandra [4], and BigTable [5].

Data partition

For large applications, it is infeasible to fit the complete data set in a single server. The simplest way to accomplish this is to split the data into smaller partitions and then store many copies of data on multiple servers.

- Distribute data across multiple server-nodes.
- Minimize data movement when nodes are added or removed.

Consistent hashing discussed in the previous chapter is a great technique to solve these problems. Let us revisit how consistent hashing works at a high level.

- First, servers are placed on a hash ring. Suppose a hash ranges, representing from x_0 to x_n, are placed on the hash ring.
- Next, a key is hashed onto the same ring, and it is stored on the first server encountered while moving in the clockwise direction. For example, assume hash(key) is between x_0 and x_1, key is stored on server x_1.

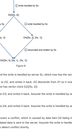

Figure 4

Using consistent hashing to partition data has the following advantages:

Automatic scaling: server nodes could be added and removed automatically depending on the load.

Heterogeneity: the number of virtual nodes for a server is proportional to the server capacity. For example, servers with higher capacity are assigned with more virtual nodes.

Data replication

To achieve high availability and reliability, data must be replicated asynchronously over N servers, where N is a configurable parameter. These N servers are chosen as follows: after a key is mapped to a server, we move clockwise on the hash ring and choose the next N servers that are located and choose the first N servers on the ring to store data copies. In Figure 5 (R1, R2, R3 represent server s1, s2, and s3).

Figure 5

With virtual nodes, the first N nodes on the ring may be owned by fewer than N physical servers. To avoid this issue, we only choose unique servers while performing the clockwise walk logic.

Nodes in the same data center often fail at the same time due to power outages, network issues, natural disasters, etc. For better reliability, replicas are placed in distinct data centers, and data centers are connected through high speed networks.

Consistency

Since data is replicated at multiple nodes, it must be synchronized across replicas. Quorum consensus can guarantee consistency for both read and write operations. Let's define a few concepts first.

N = The number of replicas

W = A write quorum of size W. For a write operation to be considered as successful, write operation must be acknowledged from W replicas.

R = A read quorum of size R. For a read operation to be considered as successful, read operation must wait for responses from R replicas.

Consider the following example shown in Figure 6 (N = 3, R = 2, W = 2).

Figure 6

If a server is not available, it takes two other servers under consideration. For example, in Figure 7, s0 is unavailable, so s1 and s2 are used to store and serve the data.

Figure 7

Configuration of N, W and R is a typical trade-off between latency and consistency. If R = 1 or W = 1, an operation is returned quickly because a coordinator only needs to wait for a response from any of the replicas. If R or W > 1, the system offers better consistency; however, the query will be slower because the coordinator must wait for the response from the slowest replica.

If R + W > N, strong consistency is guaranteed because there must be at least one overlapping node that has the latest data to ensure consistency.

How to configure N, W and R to fit our use cases? Here are some of the possible setups:

If R = 1 and W = N, the system is optimized for a fast read.

If W = 1 and R = N, the system is optimized for fast write.

If R + W > N, strong consistency is guaranteed (Usually N = 3, R = W = 2).

If R + W ≤ N, strong consistency is not guaranteed.

Depending on the requirements, we can tune the values of R, W, N to achieve the desired level of consistency.

Consistency models

Consistency model is other important factor to consider when designing a key-value store. A consistency model defines the degree of data consistency, and a wide spectrum of possible consistency models exist.

- Strong consistency: any read operation returns a value corresponding to the result of the most updated write data item. A client never sees out-of-date data.
- Weak consistency: subsequent read operations may not see the most updated value.
- Eventual consistency: this is a specific form of weak consistency. Given enough time, all updates are propagated, and all replicas are consistent.

Strong consistency is usually achieved by forcing a replica not to accept new reads/writes until every replica has agreed on current write. This approach is not ideal for highly available systems because it could block new operations. Dynamo and Cassandra offer eventual consistency, which is our recommended consistency model for our key-value store. From concurrent writes, eventual consistency allows inconsistent values to enter the system and force the client to read the values to reconcile. The rest chapter explains how reconciliation works with versioning.

In a leaderless configuration, proposed by Amazon's Dynamo [3], when data is modified, it is sent to all replicas in parallel.

In a leader-follower configuration, data is updated by sending a request to the leader. Follower nodes receive a stream of data from the leader, and the write operation is considered commited when the leader receives acknowledgements from a certain number of follower nodes.

Inconsistency resolution: versioning

Replication gives high availability but causes inconsistencies among replicas. Versioning and vector locks are used to solve inconsistency problems. Versioning means that we treat each modification as a new immutable version of data. Let's use a simple example to explain how versioning works. Assume we have a key-value replication system.

As shown in Figure 1 both replica nodes n1 and n2 have the same value: cat on n0 first value the original value. Server 1 and server 2 get the same value for get("name") operation.

Figure 1

Next, server 1 changes the name to "put(name,"puppy")", and server 2 changes the name to "put(name,"pig")" as shown in Figure 2. These two changes are performed on the replica servers without any conflicts, replica values are now different.

Figure 2

In this example, the original value could be ignored because the modifications were based on it. However, there is no clear way to resolve the conflict of the last two versions. To resolve this issue, we need a versioning system that can detect conflicts and reconcile conflicts. A vector clock is a simple data structure to resolve conflicts. Let's see how vector clocks works.

A vector clock is a server, server-pair associated with a data item. It can be used to check if one version precedes, succeeds, or in conflict with another version.

Assume a vector clock is represented by D([S1, v1], [S2, v2], ..., [Sn, vn]), where D is a data item, v1 is a version counter, and s1 is a server number, etc. If data item D is written to server Si, the system must perform one of the following tasks.

- Increment v1 if Si. v1 exists.
- Otherwise, create a new entry [Si, 1].

The above simple logic is explained with a concrete example as shown in Figure 3.

Figure 3

It's clear vector a data item D is written to server Si, it increments v1 if Si. v1 exists, otherwise, it creates a new entry for Si. This method can detect if one version precedes or succeeds another version. In addition, we can also detect if two versions are in conflict. In Figure 4, we have two clients, client X and client Y, that are trying to update the same data item. Both clients are trying to update the data item they fetched from server 1. Client X's update is successful because the version it has is the latest version. Client Y's update is rejected because the version it has is not the latest version. In this case, Client Y needs to fetch the latest version of the data item, reconcile the conflicts, and then send the update request based on the latest version.

Figure 4

If client writes a data item D to server Si, it increments v1 if Si. v1 exists. Otherwise, it creates a new entry for Si. This method can detect if one version precedes or succeeds another version. In addition, we can also detect if two versions are in conflict. In Figure 5, client X wrote to server 1, client Y wrote to server 2, client Z wrote to server 3. The system must perform one of the following tasks.

- Increment v1 if Si. v1 exists.
- Otherwise, create a new entry [Si, 1].

The above simple logic is explained with a concrete example as shown in Figure 5.

Figure 5

It is clear vector a data item D is written to server Si, it increments v1 if Si. v1 exists. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 1]). Sy, Sz are similar. Assume Sy is stored in Sx and Sz is stored in Sy. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 1], [Sy, 1]). Sz is stored in Sx. Assume Sx is stored in Sz. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 2], [Sy, 1]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 2], [Sy, 2]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 2], [Sy, 2], [Sz, 1]). Sz is stored in Sx. Assume Sx is stored in Sz. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 3], [Sy, 2], [Sz, 1]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 3], [Sy, 3], [Sz, 1]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 3], [Sy, 3], [Sz, 2]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 4], [Sy, 3], [Sz, 2]). Sz is stored in Sx. Assume Sx is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 4], [Sy, 3], [Sz, 3]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 5], [Sy, 3], [Sz, 3]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 5], [Sy, 4], [Sz, 3]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 5], [Sy, 4], [Sz, 4]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 6], [Sy, 4], [Sz, 4]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 6], [Sy, 5], [Sz, 4]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 6], [Sy, 5], [Sz, 5]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 7], [Sy, 5], [Sz, 5]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 7], [Sy, 6], [Sz, 5]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 7], [Sy, 6], [Sz, 6]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 8], [Sy, 6], [Sz, 6]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 8], [Sy, 7], [Sz, 6]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 8], [Sy, 7], [Sz, 7]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 9], [Sy, 7], [Sz, 7]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 9], [Sy, 8], [Sz, 7]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 9], [Sy, 8], [Sz, 8]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 10], [Sy, 8], [Sz, 8]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 10], [Sy, 9], [Sz, 8]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 10], [Sy, 9], [Sz, 9]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 11], [Sy, 9], [Sz, 9]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 11], [Sy, 10], [Sz, 9]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 11], [Sy, 10], [Sz, 10]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 12], [Sy, 10], [Sz, 10]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 12], [Sy, 11], [Sz, 10]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 12], [Sy, 11], [Sz, 11]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 13], [Sy, 11], [Sz, 11]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 13], [Sy, 12], [Sz, 11]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 13], [Sy, 12], [Sz, 12]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 14], [Sy, 12], [Sz, 12]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 14], [Sy, 13], [Sz, 12]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 14], [Sy, 13], [Sz, 13]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 15], [Sy, 13], [Sz, 13]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 15], [Sy, 14], [Sz, 13]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 15], [Sy, 14], [Sz, 14]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 16], [Sy, 14], [Sz, 14]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 16], [Sy, 15], [Sz, 14]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 16], [Sy, 15], [Sz, 15]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 17], [Sy, 15], [Sz, 15]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 17], [Sy, 16], [Sz, 15]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 17], [Sy, 16], [Sz, 16]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 18], [Sy, 16], [Sz, 16]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 18], [Sy, 17], [Sz, 16]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 18], [Sy, 17], [Sz, 17]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 19], [Sy, 17], [Sz, 17]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 19], [Sy, 18], [Sz, 17]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 19], [Sy, 18], [Sz, 18]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 20], [Sy, 18], [Sz, 18]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 20], [Sy, 19], [Sz, 18]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 20], [Sy, 19], [Sz, 19]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 21], [Sy, 19], [Sz, 19]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 21], [Sy, 20], [Sz, 19]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 21], [Sy, 20], [Sz, 20]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 22], [Sy, 20], [Sz, 20]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 22], [Sy, 21], [Sz, 20]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 22], [Sy, 21], [Sz, 21]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 23], [Sy, 21], [Sz, 21]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 23], [Sy, 22], [Sz, 21]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 23], [Sy, 22], [Sz, 22]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 24], [Sy, 22], [Sz, 22]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 24], [Sy, 23], [Sz, 22]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 24], [Sy, 23], [Sz, 23]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 25], [Sy, 23], [Sz, 23]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 25], [Sy, 24], [Sz, 23]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 25], [Sy, 24], [Sz, 24]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 26], [Sy, 24], [Sz, 24]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 26], [Sy, 25], [Sz, 24]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 26], [Sy, 25], [Sz, 25]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 27], [Sy, 25], [Sz, 25]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 27], [Sy, 26], [Sz, 25]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 27], [Sy, 26], [Sz, 26]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 28], [Sy, 26], [Sz, 26]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 28], [Sy, 27], [Sz, 26]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 28], [Sy, 27], [Sz, 27]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 29], [Sy, 27], [Sz, 27]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 29], [Sy, 28], [Sz, 27]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 29], [Sy, 28], [Sz, 28]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 30], [Sy, 28], [Sz, 28]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 30], [Sy, 29], [Sz, 28]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 30], [Sy, 29], [Sz, 29]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 31], [Sy, 29], [Sz, 29]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 31], [Sy, 30], [Sz, 29]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 31], [Sy, 30], [Sz, 30]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 32], [Sy, 30], [Sz, 30]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 32], [Sy, 31], [Sz, 30]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 32], [Sy, 31], [Sz, 31]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 33], [Sy, 31], [Sz, 31]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 33], [Sy, 32], [Sz, 31]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 33], [Sy, 32], [Sz, 32]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 34], [Sy, 32], [Sz, 32]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 34], [Sy, 33], [Sz, 32]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 34], [Sy, 33], [Sz, 33]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 35], [Sy, 33], [Sz, 33]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 35], [Sy, 34], [Sz, 33]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 35], [Sy, 34], [Sz, 34]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 36], [Sy, 34], [Sz, 34]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 36], [Sy, 35], [Sz, 34]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 36], [Sy, 35], [Sz, 35]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 37], [Sy, 35], [Sz, 35]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 37], [Sy, 36], [Sz, 35]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 37], [Sy, 36], [Sz, 36]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 38], [Sy, 36], [Sz, 36]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 38], [Sy, 37], [Sz, 36]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 38], [Sy, 37], [Sz, 37]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 39], [Sy, 37], [Sz, 37]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 39], [Sy, 38], [Sz, 37]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 39], [Sy, 38], [Sz, 38]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 40], [Sy, 38], [Sz, 38]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 40], [Sy, 39], [Sz, 38]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 40], [Sy, 39], [Sz, 39]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 41], [Sy, 39], [Sz, 39]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 41], [Sy, 40], [Sz, 39]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 41], [Sy, 40], [Sz, 40]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 42], [Sy, 40], [Sz, 40]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 42], [Sy, 41], [Sz, 40]). Sz is stored in Sy. Assume Sy is stored in Sz. Assume the vector is handled by Sz, which now has vector clock. D([Sx, 42], [Sy, 41], [Sz, 41]). Sx is stored in Sz. Assume Sz is stored in Sx. Assume the vector is handled by Sx, which now has vector clock. D([Sx, 43], [Sy, 41], [Sz, 41]). Sy is stored in Sx. Assume Sx is stored in Sy. Assume the vector is handled by Sy, which now has vector clock. D([Sx, 43], [Sy, 42], [Sz, 41

handling data center outage, it is important to replicate data across multiple data centers. Even if a data center is completely offline, users can still access data through the other data centers.

System architecture diagram

Now that we have discussed different technical considerations in designing a key-value store, we can shift our focus on the architecture diagram, shown in Figure 17.

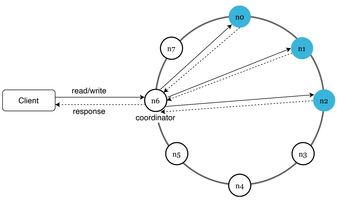

Figure 17

Main features of the architecture are listed as follows:

- Clients communicate with the key-value store through simple APIs: *get(key)* and *put(key, value)*.
- A coordinator is a node that acts as a proxy between the client and the key-value store.
- Nodes are distributed on a ring using consistent hashing.
- The system is completely decentralized so adding and moving nodes can be automatic.
- Data is replicated at multiple nodes.
- There is no single point of failure as every node has the same set of responsibilities.

As the design is decentralized, each node performs many tasks as presented in Figure 18.

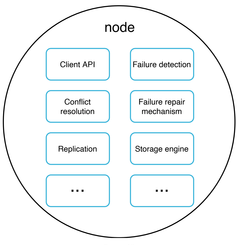

Figure 18

Write path

Figure 19 explains what happens after a write request is directed to a specific node. Please note the proposed designs for write/read paths are primary based on the architecture of Cassandra [8].

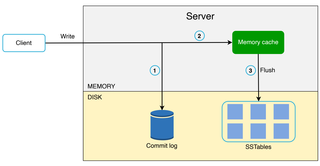

Figure 19

1. The write request is persisted on a commit log file.

2. Data is saved in the memory cache.

3. When the memory cache is full or reaches a predefined threshold, data is flushed to SSTable [9] on disk. Note: A sorted-string table (SSTable) is a sorted list of <key, value> pairs. For readers interested in learning more about SSTable, refer to the reference material [9].

Read path

After a read request is directed to a specific node, it first checks if data is in the memory cache. If so, the data is returned to the client as shown in Figure 20.

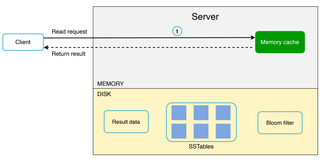

Figure 20

If the data is not in memory, it will be retrieved from the disk instead. We need an efficient way to find out which SSTable contains the key. Bloom filter [10] is commonly used to solve this problem.

The read path is shown in Figure 21 when data is not in memory.

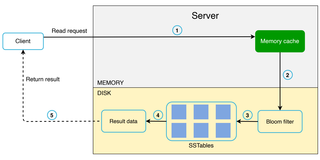

Figure 21

1. The system first checks if data is in memory. If not, go to step 2.

2. If data is not in memory, the system checks the bloom filter.

3. The bloom filter is used to figure out which SSTables might contain the key.

4. SSTables return the result of the data set.

5. The result of the data set is returned to the client.

Summary

This chapter covers many concepts and techniques. To refresh your memory, the following table summarizes features and corresponding techniques used for a distributed key-value store.

Goal/Problems	Technique
Ability to store big data	Use consistent hashing to spread load across servers
High availability reads	Data replication Multi-datacenter setup
Highly available writes	Versioning and conflict resolution with vector clocks
Dataset partition	Consistent Hashing
Incremental scalability	Consistent Hashing
Heterogeneity	Consistent Hashing
Tunable consistency	Quorum consensus
Handling temporary failures	Sloppy quorum and hinted handoff
Handling permanent failures	Merkle tree
Handling data center outage	Cross-datacenter replication

Table 2

Reference materials

[1] Amazon DynamoDB: https://aws.amazon.com/dynamodb/

[2] memcached: https://memcached.org/

[3] Redis: https://redis.io/

[4] Dynamo: Amazon's Highly Available Key-value Store: https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf

[5] Cassandra: https://cassandra.apache.org/

[6] Bigtable: A Distributed Storage System for Structured Data: https://static.googleusercontent.com/media/research.google.com/en//archive/bigtable-osdi06.pdf

[7] Merkle tree: https://en.wikipedia.org/wiki/Merkle_tree

[8] Cassandra architecture: https://cassandra.apache.org/doc/latest/architecture/

[9] SStable: https://www.igvita.com/2012/02/06/sstable-and-log-structured-storage-leveldb/

[10] Bloom filter https://en.wikipedia.org/wiki/Bloom_filter

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.